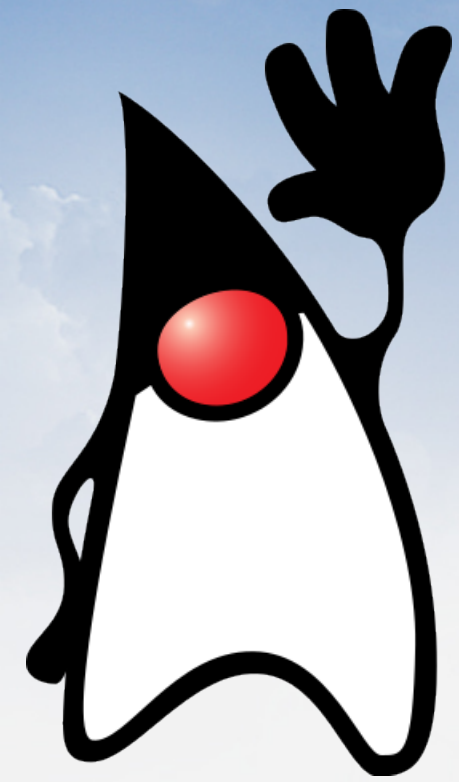




More on Objects

COP 3330
Department of Computer Science
University of Central Florida
Dr. Bacanli

Why do we need to talk more about Objects themselves?

- Hopefully by now in the course, you are starting to see the behavior of OOP concepts being applied.
- Now we want to explore other aspects of our class designs in object oriented programming.



Copy constructor

```
package carTests;
public class Car{

    private int year;
    private String brand;
    // let us consider getters and setters
    // are defined

    public Car(){
        year = 2000;
        brand = "Toyota";
    }

    public Car(Car copy){
        year = copy.getYear();
        brand = copy.getBrand();
    }

}
```

```
package carTests;
public class CarTest{

    public static void main(String[] args){
        Car mycar = new Car();
        Car copyCar = new Car(mycar);
    }

}
```

Supposing getters and setters are defined

Static vs Non-Static

- We will see static methods and instance variables inside a class
- Instance variables can be static.
- Instance methods can access static and non-static variables
- Static methods can only access static instance variables
- We had seen static variables in standard Java Library before
Math.PI and Math.E
- We had seen static methods in standard Java Library before
Math.sqrt(double a) and Math.pow(double a, double b)

Static instance(?) method

```
package carTests;

public class Car{

    private int year;
    private String brand;

    public Car(){
        year = 2000;
        brand = "Toyota";
    }

    public static void sayHi(){
        System.out.println("I am car");
    }

}
```

How to make sayHi() syntactically correct?

```
package carTests;

public class CarTest{

    public static void main(String[] args){
        Car mycar = new Car();
        mycar.sayHi(); //Syntactically correct
        Car.sayHi(); //ethically correct
        sayHi();// Syntactically incorrect
    }

}
```

Can I access instance variable inside a static method?

NO!

Static instance variable inside a Class

```
package carTests;
public class Car{

    private int year;
    private String owner;
    private static int count=0;
    private int orderNum; // this is done for example

    public Car(String owner){
        year = 2000;
        this.owner = owner;
        count++;
        orderNum=count;
    }
    //we can access count in non static instance method!
    public void sayCount(){ // if we make static,ERROR
        System.out.println(owner +" car is "+ count);
    }

    public int order(){ return orderNum; }
}
```

```
package carTests;
public class CarTest{

    public static void main(String[] args){
        Car johnCar = new Car("John");
        johnCar.sayCount();

        Car jimCar = new Car("Jim");
        jimCar.sayCount();
        System.out.println();//leave a line

        johnCar.sayCount();
        System.out.print(johnCar.order());
    }
}
```

John car is 1
Jim car is 2

John car is 2
1



Objects as instance variables



```
package bTests;
public class Bicycle{

    private int year;
    private Wheel w1,w2;

    public Bicycle(){
        year = 2000;
        System.out.println("Bicycle is created");
        w1= new Wheel();
        w2 = new Wheel();
    }

    public void go(){
        w1.turn();
        w2.turn();
        System.out.println("Bicycle is going");
    }
}
```

```
package bTests;
public class Wheel{
    private int diameter;

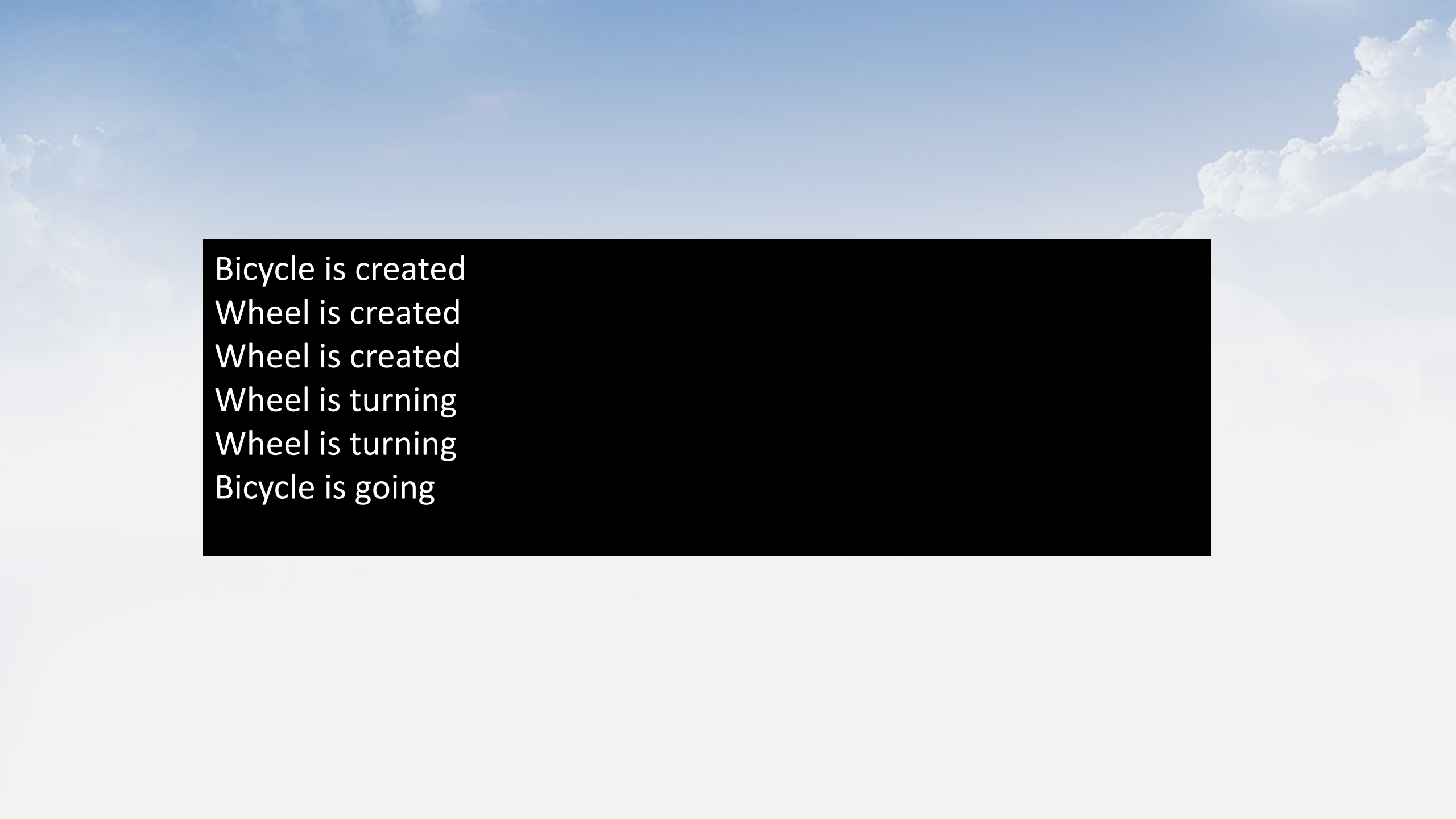
    public Wheel(){
        diameter = 20;
        System.out.println("Wheel is created");
    }

    public void turn(){
        System.out.println("Wheel is turning");
    }
}
```

```
package bTests;
public class bTester{
    public static void main(String[] args){
        Bicycle myBike = new Bicycle();
        myBike.go();
    }
}
```

Static methods and variables

- Static instance variables will be useful if we want to create order number for each created object
 - or if want to keep track of number of objects created so far
- Static instance variables can be a reference to an object
 - In that case all the objects will be aware of the updates to that object
- Static methods are useful for creating validation checking functions.
 - Static methods do not reach the instance variables. If the function is not doing something related to object, making it static will be nice.
- Static methods are also accessor functions for the static instance variables.



Bicycle is created
Wheel is created
Wheel is created
Wheel is turning
Wheel is turning
Bicycle is going



ArrayLists as instance variables



```
package bTests;
import java.util.ArrayList;
public class Bicycle{

    private ArrayList<Wheel> wheels;
    public Bicycle(int numWheels){
        wheels = new ArrayList<>();
        System.out.println("Bicycle is created");
        for(int i=0;i<numWheels;i++){
            wheels.add(new Wheel());
        }
    }

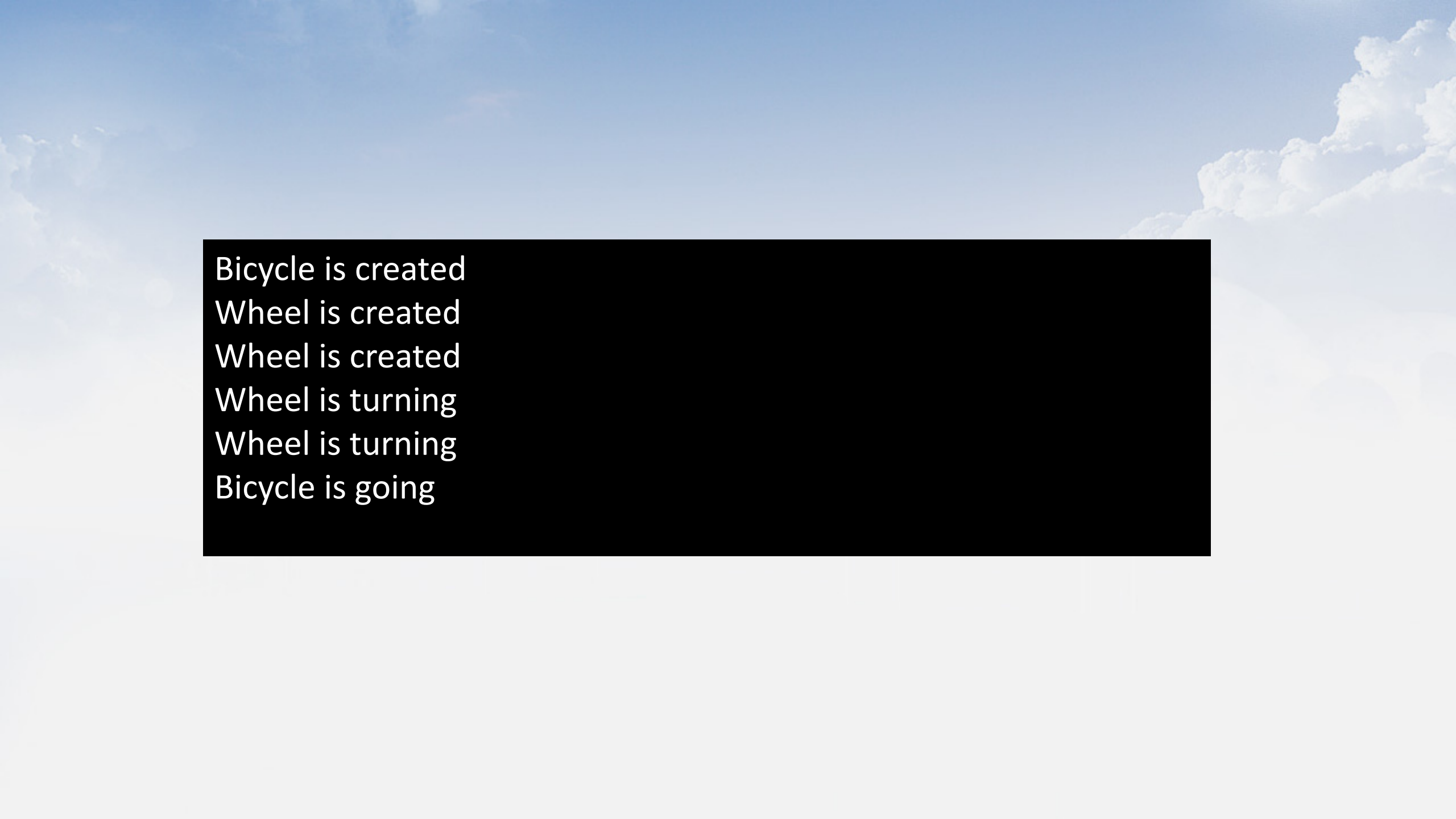
    public void go(){
        for(Wheel w: wheels){ // remember this?
            w.turn();
        }
        System.out.println("Bicycle is going");
    }
}
```

```
package bTests;
public class Wheel{
    private int diameter;

    public Wheel(){
        diameter = (int)(Math.random()*5)+20;
        System.out.println("Wheel is created");
    }

    public void turn(){
        System.out.println("Wheel is turning");
    }
}
```

```
package bTests;
public class bTester{
    public static void main(String[] args){
        Bicycle myBike = new Bicycle(2);
        myBike.go();
    }
}
```



Bicycle is created
Wheel is created
Wheel is created
Wheel is turning
Wheel is turning
Bicycle is going

Methods

- Did you notice in the Java Docs, that there were methods with the same name!
- How did Java know which method to invoke?
 - The Signature

Method Signature

- The Method Signature is the header of a Java method.
 - This is the name and parameter type.
 - It does not include return type!
- If two Methods have the same parameters but different data type it returns:
 - Java will not know which method to call.
 - This is known as ambiguous methods.
- We can create different constructors taking different number of parameters/parameter types

method signatures

```
package wheels;
public class Wheel {
    private int turningTimes;

    Wheel(){ turningTimes=20; }

    public void turn() { // method 1
        System.out.println("Wheel is turning");
    }

    public int turn(){ // method 2
        return turningTimes;
    }

    public void turn(int times){ //method 3
        System.out.println("Wheel is turning "+
            times+ " times");
    }

}

} //end of class
```

```
package wheels;
public class WheelTester{

    public static void main(String[] args){
        Wheel w = new Wheel();
        w.turn(); // 2 methods exist
        int times = w.turn();
        turn(5);
    }

} //end of class
```

Duplicate method turn in type Wheel

Either method 1 or method 2 is invalid

```
public void turn(boolean b) { // method 1
    if (b) {
        System.out.println("Wheel is turning");
    }
}
```

method signatures

```
package wheels;  
public class Wheel {  
    private int turningTimes;  
  
    Wheel(){ turningTimes=20; }  
  
    public int turn(){  
        turningTimes--;  
        return turningTimes;  
    }  
  
} //end of class
```

We are not interested in returned value. We are interested in what the method is doing in fact.

Here it reduces turningTimes. It is enough for me to reduce turningTimes. I do not need the returned value

```
package wheels;  
public class WheelTester{  
  
    public static void main(String[] args){  
        Wheel w = new Wheel();  
        w.turn();// valid, syntactically correct  
        int t_num=w.turn(); //correct  
    }  
}
```

Getting the returned value inside a variable is syntactically correct format.

this keyword

- this is a special keyword in java that refers to the current object in control of the program.
 - There maybe statements written that can cause some ambiguity between the programmer and sometimes the compiler.
 - Let's take a look at some!
- Can you call constructor from an instance method?
- Can you call constructor from a constructor?

```
package wheels;
public class Wheel {
    private int turningTimes;

    public Wheel(){
        this(4); // to call constructor
    }

    public Wheel(int t){
        turningTimes=t;
    }

    public void turn() {
        this.turn(turningTimes);
    }

    private void turn(int times){ // why private?
        if( this.turningTimes > times ){
            System.out.println( "Turning " + times
                               + " times" );
        }
    }
    public Wheel getSelf(){ return this; } // use as is
}
```

Uses of *this*

Can we write Wheel(4) instead?

```
package wheels;
public class WheelTester{

    public static void main(String[] args){
        Wheel w = new Wheel();
        Wheel w2 = w.getSelf();
        if(w.getSelf() == w) {
            System.out.println("They are the
same");
        }
        w.turn();
    } //end of main
}
```

Lessons about constructor call and this

```
package wheels;
public class Wheel {
    private int turningTimes;

    public Wheel(){
        this(4); // to call constructor OK
    }

    public Wheel(int t){
        this(); // Recursive constructor invocation Car()
    }

    public void turn() {
        Wheel(); // Compilation error
    }

    private void turn(int times){
        this(); // compilation error
    } //end of turn(int times) method
} //end of class Wheel
```

The constructor (the method which has the same name with the class) cannot be called from anywhere inside the class.

this(..) call will be calling the constructor of the class. this(...) cannot be used anywhere except constructor.

Writing *this.* keyword before any inner method/instance variable is syntactically correct but ethically not. Unless there is a confusion, do not write this.method..

Cyclic calls between constructors will be detected by the compiler.